

BioNix_Week_5_Wesley_ILP

Wednesday, 13 August 2025 15:01

Now, restart the Nix, from the most basic

<https://nixos.org/guides/nix-pills/03-enter-environment.html>

Group Questions and answers (Provided by Shreyas)

1. Put your technical diary into the WEHI-wide sharepoint/teams group folder under BioNix and then provide us with the link. This will help us find it after you are gone.
 1. I've looked at your technical diary and it is very useful!
 2. Please make sure you share it as a link in the teams's weekly project email update along with other key documentation like your ILP and everyone else's ILP and technical diary.
2. Q: Is one of the goals of being an open-source contributor in this project to improve the documentation around BioNix? I have been referring to the GitHub pages and the public project page.
 1. Kind of, but really we need to get people up to speed with Nix first, before we can get them to think about BioNix.
 2. Ideally we would like you to contribute to next packages and to look at migrating a bash script into BioNix if you can get that far.
3. Q: Will our technical diaries contribute to improving the documentation and help make it easier for new researchers to learn about Nix and BioNix?
 1. Technical Diaries are more like logs so they are there to log all the things that you're doing and they are supposed to be quite raw.
 2. If you have something that you wanted to document properly you might need to go back through your technical diary, copy it over to a new document, and clean it up so that it's easy for someone to follow.
 3. You would then need to give it to someone naive to test if your documentation is accurate.
 4. They would then clean up your documentation to make sure it is easy for a naive user to follow.
4. Q: Could you also confirm if learning through this process and documenting the errors I encounter, along with any added instructions or debugging steps, would be a sensible and useful contribution?
 1. This would be very useful to put into your technical diary however someone might be using a slightly different windows version or they might be using a completely different operating system so it does have limitations to a certain extent.
 2. This would still be beneficial both for you and for future interns if you had a good technical diary and a good individual learning plan which might have more links to higher level youtube videos and documentation and a summary of what you've learned from those resources.

Here's the detailed summary of "Why You Should Give it a Try" from the *Nix in pills* series:

1. Purpose of the Post

- Goal: Convince readers to try Nix, a purely functional package manager and deployment system for POSIX systems.
- Scope: No need to install NixOS; examples will use NixOS where relevant.
- Motivation:
 - Existing Nix/NixOS documentation is good but can feel “magical” for newcomers.
 - This series aims to provide complementary, short, digestible explanations (“pills”).

2. Problem with Traditional Package Managers

- Mutable global state:
 - Managers like dpkg and rpm overwrite system locations (e.g., /usr/bin/foo).
 - Installing multiple versions (e.g., foo-1.0 and foo-1.1) is painful.
- Workarounds exist but are inconvenient:
 - Sysadmin approach: Containers for different versions → adds complexity (orchestration, monitoring).
 - Developer approach: Tools like virtualenv or jhbuild help but cause duplication and system library conflicts.
- Key limitation: Hard to mix and share multiple software stacks (e.g., GNOME 3.10 & 3.12).

3. Nix's Purely Functional Model

- No assumption of global state:
 - All software stored in /nix/store as immutable, versioned paths: /nix/store/<hash>-<name>-

<version>/.

- Multiple versions can coexist (e.g., MySQL 5.2 with glibc 2.18, MySQL 5.5 with glibc 2.19).
- Derivation concept:
 - “Derivation” is Nix’s build artifact (like a package, but hash-identified).
 - Dependencies are explicit; no dependency hell or resolution algorithm needed.
- Advantages:
 - Maximum flexibility in mixing versions.
 - Predictable and reproducible builds.

4. Immutable vs. Mutable Trade-offs

- Immutable builds:
 - Upgrading a library (e.g., glibc) requires recompiling dependent software (path hardcoded).
 - Security updates require special handling (still pure).
- Runtime composition challenges:
 - No global plugin paths; apps like Firefox need wrapping to locate plugins.
- No formal upgrade/downgrade:
 - Switching versions means switching to an entirely new derivation; user handles data migration if formats change.

5. Conclusion & Value Proposition

- Strengths:
 - Flexible build-time composition.
 - Highly reproducible builds.
 - Reliable, consistent cloud deployment (NixOps makes many orchestration tools obsolete).
- Weaknesses:
 - Dynamic runtime composition is harder.
 - Replacing core libraries requires rebuilds.
- Maturity:
 - Nixpkgs is an actively developed, non-experimental repository with growing contributions.
 - Worth investing time in learning.

Entering and Using the Nix Environment

- Learn how to make nix-env, nix-store, and other commands available in your shell (via source ~/.nix-profile/etc/profile.d/nix.sh or multi-user mode daemon script).
- Understand that the Nix user environment is separate from the system-wide environment.

Installing Software and Managing the User Environment

- Install a program with nix-env -i hello — applies only to the current Nix user.
- Installation creates a new user environment → essentially a “snapshot.”
- Use nix-env --list-generations to view different generations.
- Use nix-env -q to list installed packages.

Understanding Profile Path Merging

- Executables in ~/.nix-profile/bin are symlinks to /nix/store/....
- Nix merges all bin paths from your installed derivations into the profile.

Rolling Back and Switching Generations

- nix-env --rollback to go back to the previous generation.
- nix-env -G <n> to switch to a specific generation.
- No need to reinstall like in traditional package managers — switching is instant.

Querying the Store and Dependencies

- nix-store -q --references shows a derivation’s direct dependencies.
- nix-store -q --referrers shows what depends on it (reverse dependencies).
- nix-store -qR shows the closure (all recursive dependencies).
- nix-store -q --tree visualizes the dependency tree.
- Helps you understand that the Nix store contains all build outputs, and the environment is just a

“view” pointing to them.

No Dependency Resolution

- Nix has no complex SAT dependency solving like apt; each derivation’s dependencies are fixed and immutable.
- No “version bound” conflicts — each combination is its own derivation.

Recovering the Environment

- Even if you accidentally remove Nix from the environment with `nix-env -e '*'`, you can find its binary in the store and run `rollback` or `reinstall` from there.

Managing Software Sources with Channels

- Use `nix-channel --list` to view channels.
- Use `nix-channel --update` to fetch the latest package definitions (similar to `apt-get update`).

Core Significance

- Moves you from “knowing how to install Nix” to “knowing how to use Nix to manage your software environment.”
- Teaches essential skills: `install`, `rollback`, `switch`, `uninstall`, `upgrade`.
- Builds understanding of the profile ↔ store ↔ derivation relationship, and how this differs from traditional package managers.
- Lays the foundation for learning the Nix language by first understanding derivations and dependencies.

"man" code = **manual pages viewer**

Eg: in bash

`"man hello"`

Which gives

```
HELLO(1)                                User Commands                                HELLO(1)
```

```
NAME
  hello - friendly greeting program
```

```
SYNOPSIS
  hello [OPTION]...
```

```
DESCRIPTION
  Print a friendly, customizable greeting.
```

```
  -t, --traditional
        use traditional greeting
```

```
  -g, --greeting=TEXT
        use TEXT as the greeting message
```

```
  --help display this help and exit
```

```
  --version
        output version information and exit
```

```
ADDITIONAL INFORMATION
  See man/hello.x <https://git.savannah.gnu.org/cgiit/hello.git/tree/man/hello.x> for how to
  add additional information to a manual page.
```

```
AUTHOR
  Written by Karl Berry, Sami Kerola, Jim Meyering, and Reuben Thomas.
```

```
REPORTING BUGS
  Report bugs to: bug-hello@gnu.org
  GNU Hello home page: <https://www.gnu.org/software/hello/>
  General help using GNU software: <https://www.gnu.org/gethelp/>
  Report GNU Hello translation bugs to <https://translationproject.org/team/>
```

COPYRIGHT

Copyright © 2025 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later
<<https://gnu.org/licenses/gpl.html>>.
This is free software: you are free to change and redistribute it. There is NO WARRANTY,
to the extent permitted by law.

SEE ALSO

The full documentation for **hello** is maintained as a Texinfo manual. If the **info** and **hello** programs are properly installed at your site, the command

info hello

should give you access to the complete manual.

GNU Hello 2.12.2

May 2025

HELLO(1)

~

(END)

Press q for exit

"nix-env -i man-db"

Eg: "nix-env -iA nixpkgs.man-db"

- Nix in "-env" environment
- "iA"
 - o "-i" install the package by attribute name "A"
- nixpkgs.man-db → the attribute path inside the Nixpkgs package set that refers to the man-db package.

man-db

- "man" the package name
- "-db" database

common symbols in docs

- "\$"
 - o Meaning: Standard shell prompt for a regular (non-root) user.
 - o It's the shell prompt symbol
 - o Example in docs:
 - \$ ls -l
 - "ls" list command
 - "-l" long format option
 - ◆ Displays extra details like:
 - file type (directory, symlink, etc.)
 - permissions (rwx)
 - number of links
 - owner and group
 - file size
 - modification date
 - file name (and symlink target if any).
- "#"
 - Meaning: Shell prompt when running as root (superuser).
 - Example in docs:
 - # apt-get update
 - "apt-get" the package manager command for Debian/Ubuntu systems.
 - ◆ Debian = a stable, community Linux distribution.
 - ◆ Ubuntu = a popular, user-friendly Linux distribution based on Debian.
 - ◆ Both use apt-get for package management.
 - "Update" tells apt-get to refresh the local package index.
- %
 - Meaning: Often used in tcsh, csh, or sometimes zsh as the default prompt.
 - Example:
 - % echo "hello"
 - "Echo" a built-in shell command that prints text to the terminal (standard output).
- >
 - Meaning: Usually shows line continuation in documentation (rest part of command)

- meaning: Usually shows line continuation in documentation (not part of command).
- Example:
 - `$ [ -f /nix/.../nix-daemon.sh ] && \`
`> source /nix/.../nix-daemon.sh`
 - `"-f"` checks if a file exists and is a regular file.
 - `&&` Means: only run the next command if the previous one succeeded (true).

nix-env --rollback

Meaning of `--rollback`

- `--rollback` = *switch back to the previous generation of your user environment.*
- In Nix, every time you install or remove a package with `nix-env`, it creates a new generation (a snapshot of your environment).
- `--rollback` discards the current generation and reactivates the one before it.

Eg:

```
MacBookPro:~ wesley$ nix-env -iA nixpkgs.hello
replacing old 'hello-2.12.2'
installing 'hello-2.12.2'
building '/nix/store/vhwdxdf7mzlsyywzlx9kjsxr0jr4qf0v-user-environment.drv'...
MacBookPro:~ wesley$ nix-env -iA nixpkgs.cowsay
replacing old 'cowsay-3.8.4'
installing 'cowsay-3.8.4'
MacBookPro:~ wesley$ nix-env --rollback
switching profile from version 5 to 4
MacBookPro:~ wesley$
```

→ Switches back from generation 5 → generation 4.
 (so now cowsay disappears, only hello remains).

nix-env -q

- `"-q"` query
- “query my profile” — list what’s installed
 - `t` tells `nix-env` to *show information about your profile.*
 - By default, `nix-env -q` lists all the packages currently installed in your user environment.
- Eg:


```
MacBookPro:~ wesley$ nix-env -q
cowsay-3.8.4
hello-2.12.2
```
- Variants of `-q`
 - `nix-env -q --installed` → same as default (list installed packages).
 - `nix-env -q --available` → list packages available in your channels.
 - `nix-env -qaP` → query available packages, with attribute paths.
 - `nix-env -q hello` → search for hello in your installed environment.

nix-env -G 3

- `"-G <n>"` the `<n>` should be a number
 - switch to a specific generation `<n>` of your user environment.
 - It’s similar to `--rollback`, but instead of going back just one step, you can jump directly to any numbered generation.
- Eg:


```
MacBookPro:~ wesley$ nix-env -iA nixpkgs.cowsay
replacing old 'cowsay-3.8.4'
installing 'cowsay-3.8.4'
MacBookPro:~ wesley$ nix-env --rollback
switching profile from version 5 to 4
MacBookPro:~ wesley$ nix-env -q
cowsay-3.8.4
hello-2.12.2
MacBookPro:~ wesley$ nix-env -G 5
switching profile from version 4 to 5
```
- It more

nix-store

- `nix-store` is the low-level command that lets you interact directly with the Nix store.
- What can `"nix-store"` do

What can nix-store do

- Query dependencies
 - `nix-store -q --references <path>` → show what a derivation depends on.
 - `nix-store -q --referrers <path>` → show what depends *on it*.
 - `nix-store -qR <path>` → show the full closure (all recursive dependencies).
 - `"-qR"` query recursively.
 - `nix-store -qR <path>` = “show me everything this store path needs to run, including indirect dependencies.”
 - ◆ Eg:

```
MacBookPro:~ wesley$ nix-store -q --references 'which hello'
/nix/store/8xi6xc6hgs7saynm4xnlpdl99k37ddfi-libiconv-109
MacBookPro:~ wesley$ nix-store -qR $(which hello)
/nix/store/8xi6xc6hgs7saynm4xnlpdl99k37ddfi-libiconv-109
/nix/store/sx6ihl7jw3lzx27a6p66a095kpbk9m7-hello-2.12.2
```
- `nix-store -q --tree <path>` → display a tree of dependencies.
 - `"--tree"`. It is a query option that tells nix-store to show dependencies as a tree structure.
 - Instead of listing dependencies in a flat list (like `-qR`), it shows them in a hierarchical format, so you can see which package depends on which.
 - Eg:

```
MacBookPro:~ wesley$ nix-store -q --tree $(which hello)
/nix/store/sx6ihl7jw3lzx27a6p66a095kpbk9m7-hello-2.12.2
├── /nix/store/8xi6xc6hgs7saynm4xnlpdl99k37ddfi-libiconv-109
└── /nix/store/8xi6xc6hgs7saynm4xnlpdl99k37ddfi-libiconv-109 [...]
```
- Realize builds
 - `nix-store --realise <drv>` → build a derivation (usually done behind the scenes).
 - `"--realise"` actually build or fetch the outputs of a derivation (.drv file).
 - ◆ A derivation = Nix's low-level build instruction (like a recipe telling Nix how to build a package).
 - ◆ By default, when you install a package with `nix-env`, Nix automatically realises the derivations in the background.
 - ◆ Eg:

```
MacBookPro:~ wesley$ nix-instantiate '<nixpkgs>' -A hello
warning: you did not specify '--add-root'; the result might be removed
by the garbage collector
/nix/store/5pb5mpkdbk1yp70as187fydi36l4ixr2-hello-2.12.2.drv
MacBookPro:~ wesley$ nix-store --realise /nix/store/abcd1234-
hello-2.12.2.drv
error: 'abcd1234-hello-2.12.2.drv' is too short to be a valid store
path
MacBookPro:~ wesley$
```

Analysis

`warning: you did not specify '--add-root'; the result might be removed by the garbage collector`

Why warning?

- When you build something with `nix-instantiate` + `nix-store --realise`, Nix puts the result in `/nix/store/....`
 - ▶ `"nix-instantiate"` the command that evaluates a Nix expression and produces a derivation (.drv file).
 - A derivation is a low-level build recipe in Nix — it describes *how* to build a package (inputs, outputs, dependencies).
 - `nix-instantiate` does not build anything.
 - It only creates the recipe (.drv) and places it in `/nix/store/.`
 - `nix-instantiate` = “translate a Nix expression into a derivation file.” It's the *planning step* before the actual build (`--realise`).
 - Eg:

```
MacBookPro:~ wesley$ nix-instantiate '<nixpkgs>' -A
hello
warning: you did not specify '--add-root'; the result
might be removed by the garbage collector
/nix/store/5pb5mpkdbk1yp70as187fydi36l4ixr2-
hello-2.12.2.drv
```
- But unless you create a GC root, Nix considers it *garbage* once nothing references it anymore.

- The warning is reminding you: “*this build output may be cleaned up later by nix-collect-garbage.*”

What’s a GC root?

- GC = garbage collector (cleans unused store paths).
- A root is a symlink that “pins” the store path so it won’t be deleted.

Fix

```
MacBookPro:~ wesley$ drv="$(nix-instantiate '<nixpkgs>' -A hello)"
warning: you did not specify '--add-root'; the result might be removed
by the garbage collector
MacBookPro:~ wesley$ out="$(nix-store --realise "$drv")"
warning: you did not specify '--add-root'; the result might be removed
by the garbage collector
MacBookPro:~ wesley$ "$out/bin/hello"
Hello, world!
```

- Garbage collection

- `nix-store --gc` → clean up unused store paths (not referenced by any profile).

- Eg:

```
MacBookPro:~ wesley$ nix-store -q --references `which hello`
/nix/store/8xi6xc6hgs7saynm4xnlpd199k37ddfi-libiconv-109
```

Q:

1. I understand that "nix-env -G <n>" can restore any previous generation. How does this compare in reliability with using "nix-shell" + pinned channels for reproducible environments? Is one preferred for research workflows at WEHI?
2. Using "nix-store -q --tree", I can see the full dependency closure of a package. In practice, how should I decide whether to keep multiple versions in the store versus cleaning them up with garbage collection?
3. I’ve read that every package is built from a derivation (.drv). Would it be useful for me to start by writing a very simple derivation (e.g. for a shell script) as practice, or should I move directly toward migrating a real BioNix bash script?
4. Right now my diary is very command-oriented (install, rollback, query). Would it be more valuable if I also documented *why* certain commands are preferred in a research setting (e.g., nix-shell vs nix-env), or should I stay focused on raw logs for now?
5. Since I now understand nix-env and nix-store, what would be a realistic first contribution to BioNix? Would it be better to start with packaging a small external tool, or attempting to convert a real pipeline script into a Nix expression?